08 · Frontend and API reference

Written by Bideci (frontend and infrastructure) for the judges. State: the version tagged `submission-20oct` in both repositories (live since 20 Oct 2026, 12:40 Dubai for the first deployment; the checks below were made on the backend `8f21177` and the frontend `7bb7ec2`). Re-checked against the live servers that day: `eval/deploy_check.py` on the live API (price mode `snapshot`) passed 71 of 71 checks; demo builds took 5 to 7 s on saved prices (live, 20 Oct; 20 interest areas: 5.4 s), a heavy 60-year goal build 12 to 17 s on the development PC; every trade check answered in under 4.5 s.

Live: website https://www.egoquant.app (Next.js on Vercel), API https://web-production-59960.up.railway.app (FastAPI on Railway), database and sign-in: Supabase (Postgres with Row-Level Security).

---

1. The pages

In plain words. EgoQuant is a guided path of five steps, then a report you can question. You make an account, answer a validated risk questionnaire, play a short risk game, choose what kinds of investments you want, and get a portfolio with reasons, stress tests, a goal simulation and a news briefing. An AI advisor sits next to the report and knows which part of it you are looking at. Nothing is ever bought; it is an illustration.

Page	What it does	Code
`/` Home	What EgoQuant is, the five steps, sign up or log in.	`src/app/page.tsx`, `src/components/welcome/*`
`/account`	Log in, sign up (step by step, with consents), then the account: saved reports, IBKR data, agreements, activity log, delete my data / account.	`src/app/account/page.tsx`, `src/components/account-*.tsx`
`/questionnaire` (I. Profile)	The risk questionnaire, one question per screen; a half-answered one survives a refresh. Answers are copied to the account.	`src/app/questionnaire/page.tsx`, `src/lib/egoquant.ts` (`syncWithAccount`)
`/game` (II. Risk Game)	Seven rounds, each a choice among six gambles, and a crash test; one third of the attitude-to-risk part of the category (`docs/RISK_GAME_BLEND.md`).	`src/app/game/page.tsx`
`/interests` (III. Investments)	Where you trade, then the kinds of investments your category allows (others greyed with the reason); optional IBKR history.	`src/app/interests/page.tsx`, `src/lib/interest-choices.ts`
`/portfolio` (IV. Portfolio)	Goal and cash needs, then the report in tabs: Summary, Goal, Outlook, Holdings, Limits, Stress, Trade check, News, Data, Method, Why. The Advisor panel is beside it.	`src/app/portfolio/page.tsx`, `src/components/report-sections.tsx`, `report-view.tsx`, `goal-view.tsx`, `briefing-view.tsx`
`/review` (V. Review)	Read the plan once more and confirm it; a confirmed approved plan can be sealed on-chain.	`src/app/review/page.tsx`, `src/components/confirm-plan.tsx`
`/advisor`	The full chat with the Advisor (streams, what-if simulations, trade checks from the saved history).	`src/app/advisor/page.tsx`
`/account/reports/[id]`	A saved report, the same tabs as the Portfolio page.	`src/app/account/reports/[id]/*`
`/verify`	Anyone can check a sealed report (.html) or a seal file (.json) against the Sepolia transaction; the file is hashed in the browser.	`src/app/verify/page.tsx`, `src/lib/proof.ts`

Page	What it does	Code
`/debugmodels`	Team-only model view; a 404 unless `EGOQUANT_DEBUG_PAGES=on` (off in production).	`src/lib/debug-pages.ts`

UI rules the team follows (black and white theme, red only for problems, tab order, wording): `docs/UI_DECISIONS.md` in the frontend repo.

2. How the browser talks to the backend

In plain words. The browser never talks to the backend or the database directly with a password or a token. It talks only to our own website, which passes each request on and adds the signed-in person's token on the server side. The token lives in a cookie that page scripts cannot read.

- **One proxy route per backend call** under `src/app/api/**/route.ts` (31 routes). `forwardToBackend` (`src/lib/backend.ts`) sends the call to `BACKEND_URL` with a time limit per route (for example 115 s for a portfolio build, `src/lib/build-timing.ts`) and returns the answer with `Content-Type: application/json` and, on a 429 or 503, the backend's `Retry-After`.
- **Sign-in cookies.** `src/lib/server-auth.ts`: the access and refresh tokens are `HttpOnly`, `SameSite=Lax`, `Secure` in production, path `/`; the access cookie lasts at most 24 h, the refresh cookie 7 days. An expired access token is renewed once and the call retried; a 401 clears both cookies.
- **Origin check.** `sameOrigin()` (`src/lib/server-auth.ts`) refuses a state-changing call whose `Sec-Fetch-Site` is not `same-origin` or whose `Origin` host differs from the site's host (403 "This request was blocked."). Every proxy that changes something runs it: the sign-in, sign-up and account proxies (`src/app/api/auth/*`, `src/app/api/account/[[...path]]`, `src/lib/anchoring-proxy.ts`) and every other `POST` route (portfolio, profile, preferences, trade check, IBKR check, chat, briefing). The only `POST` route without it is `/api/debugmodels`, which is a 404 in production.
- **Body limits.** Each proxy reads at most a fixed size (`readLimitedBody`, for example 200 kB for chat); the backend refuses larger bodies too (section 4).
- **Errors in words.** `src/lib/http-errors.ts` turns statuses into one sentence each (413 too large, 429 "try again in N seconds", 504 too slow, 401 "signed out"); the backend's own `detail` sentence is shown otherwise.

3. API reference (backend)

In plain words. These are all the backend's doors. Doors marked "account" need a signed-in person and only ever show or change that person's own rows (Row-Level Security in the database enforces it a second time). The others work without an account and keep nothing.

Auth: **none** = public; **account** = `Authorization: Bearer <Supabase access token>` (the website adds it), refused with 401. Request and response are the main fields; the full schema is `/openapi.json` on the live API.

Questionnaire, profile and preferences

Method and path	Auth	Request → response
`GET /questionnaire`	none	→ the questions (versioned, validated items quoted verbatim)
`POST /profile`	none	`AnswersIn` (answers, optional Risk Game) → category C1–C5, limits and the plain-language explanation
`POST /preferences/form`	none	answers + trading platforms → the investment-type questions for this profile (unavailable options with a reason)

Method and path	Auth	Request → response
`POST /preferences/areas`	none	answers + choices → the interest areas they map to (and the 8-area limit)
`GET /portfolio/options`	none	→ interest areas, trading platforms, data mode

Portfolio, simulations, news

Method and path	Auth	Request → response
`POST /portfolio`	none	`PortfolioIn` (answers, preferences, goal, horizon, cash needs, optional IBKR) → `PortfolioOut` : verdict, report, goal simulation, sealed `chat_context` / `what_if_context` / `briefing_context` , HTML
`POST /account/portfolio`	account	same as `/portfolio` , using the trades saved in the account; the report is saved (180 days)
`POST /portfolio/ibkr-check`	none	an IBKR report or Flex token → what could be read (nothing stored)
`POST /what-if`	account	a sealed what-if context + scenario → the re-simulated result (the Risk Engine re-checks it; `blocked` gives no numbers)
`POST /trade-check`	none	portfolio facts + one planned trade + the person's IBKR history (a Flex report or a Flex token, used once, nothing stored; without it, or with `current_holdings` , a 422) → verdict code (`advisable` / `allowed_with_warnings` / `not_advisable` , shown to the person as "within your limits", "within your limits, with warnings", "outside your limits") with reasons
`POST /account/trade-check`	account	the same on the saved history; the check is saved
`POST /account/trade-checks/{id}/decision`	account	"I made / did not make this trade"
`POST /briefing/start`	none	sealed `briefing_context` → a job id (rates every headline outside the request)
`GET /briefing/{job_id}`	none	→ progress, then the week and month briefing
`POST /briefing/{job_id}/report`	none	→ the final briefing for a report
`GET /data/status`	none	→ running settings (price mode, budgets, news worker), no secrets

Advisor chat

Method and path	Auth	Request → response
`POST /chat`	none / account	`ChatRequest` : last 30 messages, answers, sealed contexts, `page` , `view` (the report tab on screen, fixed ids only) → one reply

Method and path	Auth	Request → response
`POST /chat/stream`	none / account	the same as server-sent events (meta, delta…, done or error); every sentence passes the compliance guard before it is sent
`GET /privacy-log/{session_id}`	account	→ counts of what the redaction gateway removed in this chat session (no content)

Account

Method and path	Auth	Request → response
`GET /account/me`, `GET /account/profile`	account	→ username, name, picture, consent version
`GET /account/username-available`	none	`?name=` → `{available}`
`GET /account/consent-version`	none	→ the current consent text version
`POST /account/consent`, `POST /account/signup-consents`	account	scope + version → recorded consent
`PUT /account/avatar`	account	a preset or a small JPEG/PNG/WebP (magic bytes checked, never SVG)
`GET` / `PUT /account/onboarding`	account	the onboarding answers kept in the account
`POST /account/trades/import`, `GET /account/trades`	account	IBKR report → trades and cash flows; list them
`GET /account/reports`, `GET` / `DELETE /account/reports/{id}`	account	saved reports (latest 50)
`POST /account/reports/{id}/briefing`	account	store the final news in a saved report (refused once sealed)
`POST /account/reports/{id}/viewed`, `GET …/contexts`, `GET …/download`	account	log a view; re-sealed advisor contexts; the HTML report (for a sealed plan: the ONE sealed report, document plus seal)
`POST /account/reports/{id}/confirm`	account	confirm an approved plan; starts the on-chain seal
`GET /account/reports/{id}/seal`, `GET …/proof`	account	seal status; the technical seal file (`egoq-proof-2` , carries the sealed document)
`GET /account/seals`	account	My seals: each confirmed report with its block and transaction, newest first
`GET /account/log`	account	the person's activity log (latest 200)
`DELETE /account/data`, `DELETE /account`	account	erase the person's data tables (the picture is cleared; the sign-in and the username stay until the account is deleted); delete the account (everything goes)

Verification and health

Method and path	Auth	Request → response
`POST /verify`	none	a seal file → recomputes the commitment and compares it with the transaction (the page also hashes the sealed document itself in the browser)

Method and path	Auth	Request → response
`GET /verify/tx/{hash}`	none	→ the Sepolia transaction's data
`GET /anchoring/status`	none	→ whether sealing is on and the chain is reachable
`GET /health`, `GET /`	none	→ ok

4. Limits

In plain words. Every door has a size limit and the expensive ones have a speed limit, so one visitor cannot slow the site down for the others.

Limit	Value	Code
Request body	1 MB by default; 512 kB for chat, briefing and what-if; others per route; 413 / 411 before anything runs	`main.py BodySizeLimit`
Chat	120 requests a minute (whole service)	`main.py chat_rate_limit`
Portfolio builds and trade checks	20 a minute (whole service); 30 IBKR checks a minute	`egoquant_routes.py _check_portfolio_rate`
Account calls	30 a minute per person	`egoquant/accounts/routes.py _USER_LIMIT`
Build time	price fetching stops after 6 s for optional funds; news inside the build at most 5 s; the website waits up to 115 s	`egoquant_routes.py`, `src/lib/build-timing.ts`
Interest areas	the first 8 are used for prices and news (the report says which were left out)	`PORTFOLIO_MAX_INTEREST_AREAS`
Builds at the same time	1 (`EGOQUANT_MAX_CONCURRENT_BUILDS`); the next ones wait their turn, then get 503 "The service is busy building other portfolios" with `Retry-After: 30`	`egoquant_routes.py _build_slots`, `egoquant/settings.py`

Every 429 carries `Retry-After` (60 s), and the website turns it into "try again in N seconds" (`src/lib/http-errors.ts`). A refused or expired token is a 401 ("Please sign in again"), never a 403. One portfolio is built at a time per server process (default 1); the others wait their turn. Through the website the wait ends after 115 s with the website's own "too slow" answer (`build-timing.ts` `BUILD_PROXY_TIMEOUT_MS` = 115 000). A direct API call gets, after 120 s, a 503 "The service is busy building other portfolios" with `Retry-After: 30`. Worst local test: 20 of the heaviest 60-year builds in the same second: 8 to 11 served, the rest turned away; ordinary builds (5 to 7 s on saved prices) were not flood-tested; no gated run crashed. Why: the server has 1 GB of memory (Railway plan limit) and each extra simultaneous build adds about 150 MB (a heavy build alone peaks at 300 to 415 MB for a 10-year goal on the development PC). `GET /data/status` also lists the numerical libraries the server runs (`libraries`), so a slow or failed build can be tied to a version.

5. Security headers and their honest gaps

In plain words. The website tells browsers not to let other sites frame it, not to guess file types, and to load code only from itself. One allowance is still too wide; it is listed here rather than hidden.

Website (`next.config.ts` , every path): `X-Content-Type-Options: nosniff` , `X-Frame-Options: DENY` , `Referrer-Policy: strict-origin-when-cross-origin` , `Permissions-Policy: camera=(), microphone=(), geolocation=()` , and `Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; img-src 'self' data: blob:; connect-src 'self'; font-src 'self'; object-src 'none'; base-uri 'self'; form-action 'self'; frame-ancestors 'none'` (`'unsafe-eval'` is added only on the development server). Backend (`main.py`): the same four headers plus `Strict-Transport-Security: max-age=63072000; includeSubDomains` on every answer; CORS only for the website's origin (`CORS_ORIGINS`).

Gaps: 1. `script-src` still allows `'unsafe-inline'` : Next.js inlines its bootstrap scripts; a per-request nonce would replace it. (`'unsafe-eval'` is gone in production; checked on the live site 20 Oct.) 2. `connect-src 'self'` is strict: the browser can reach only our own site; all backend and database calls go through the proxy. 3. HSTS: Vercel sends it on the website's own address too (`max-age=63072000` ; on `*.vercel.app` also `includeSubDomains; preload`); the backend sends its own.

6. Database (Supabase) in one paragraph

In plain words. Each person's data sits in rows that the database itself hides from everyone else, and the public key of the website can read none of it.

All 12 tables have Row-Level Security on; the public (anon) role can read no table (tested 20 Oct: every table answers 401 to the public key); signed-in users may insert only listed columns (so they cannot choose a retention date or an id) and may not rewrite verdicts or seals. Every row expires after 180 days (a nightly `pg_cron` job purges them); "delete my data" erases the data tables and clears the picture, while the sign-in and the username stay until the account itself is deleted. Proofs: `supabase/tests/rls_isolation.sql` (passed on the live database on 19 Oct and again on 20 Oct) and `rls_onchain.sql` (passed on 20 Oct, again after migration 008); details and counts in `docs/supabase/SUPABASE_LOG.md` .